Signs of Significance: Predicting the Met's Important Works

Team 1: Molly Stark (mollysta@umich.edu), Sarah Moore (sarmo@umich.edu), and Sam Cohen (cohesa@umich.edu)
Github: https://github.com/mollykstark/SIADS-Milestone-II

Overview

Background

Museum curators face the complex task of selecting from vast and varied collections often numbering in the tens or hundreds of thousands which pieces to highlight in their exhibitions. Determining which artworks are significant – that is, which warrant special attention – is labor-intensive. At the Metropolitan Museum of Art in New York, significance is implicitly encoded through inclusion in features such as the "timeline of art history" and collection "highlight." These labels, however, are applied to only a small minority of items, and the criteria for selecting them are opaque.

Our project aims to scale the curatorial challenge of identifying key artworks in the Met's collection by developing a predictive model that surfaces the objects that are most likely to be considered significant, based on the Met's existing items' labels. We construct a target variable that synthesizes the two existing institutional labels mentioned above: in the timeline of art history and collection highlight. Using supervised and unsupervised learning techniques, we explore how different features in the Met's Open Access collection correlate with our target variable. Our project seeks to achieve two goals:

1. **Reveal patterns that underlie the institution's decisions** by identifying features that most strongly correlate with significant works, shedding light on opaque and subjective decision-making.
2. **Support curators in evaluating new acquisitions** by surfacing items whose attributes suggest they are significant.

Results

We evaluated several supervised and unsupervised models. Because our target feature is heavily imbalanced, we also evaluated different resampling techniques for the supervised models. Of these, the logistic regression (LR) classifier paired with SMOTE performed best, achieving the highest PR-AUC score, the second-highest F1 score, and strong recall.

We explored the structure of our data using clustering and discovered that the art – important and otherwise, falls into two large, distinct groups, and the target is distributed proportionally in those groups. The two clusters with the highest number of significant artwork had distinguishable characteristics. The first group primarily consisted of drawings and photographs produced in Western Europe during the Georgian to Victorian eras. This cluster accounted for 32% of the significant artworks. The second group is composed mostly of sculptures and costumes from East Asia during the Ming dynasty in China and the Sengoku period in Japan. This cluster accounted for 29% of the significant artworks.

Related Work

This work builds on Molly and Sarah's Milestone 1 project, which analyzed the distribution of countries of origin across the collections of the Met and the Art Institute of Chicago (AIC). Their analysis used the Met's Open Access data alongside the AIC's collection data, both of which include geographical information but lack explicit country of origin fields. To address this gap, Molly and Sarah developed a method to infer the most likely country of origin for each artwork. This allowed them to examine how different regions were represented in each institution's holdings.

Beyond our own work, the Met's dataset has been used widely in machine learning projects, primarily focused on image analysis, metadata extrapolation, and object classification for factual labels. In a supervised classification project, Sarath Sattiraju trained a random forest classifier and an SGDClassifier to predict an artwork's class across multiple target values from the original dataset. His model performed best with the random forest classifier, achieving an accuracy of 85.7%. (Sattiraju 2018). This multi-class classification problem utilizes a much smaller subset of the Met's data and addresses relatively balanced target variables. By contrast, our project is a binary classification problem with extreme class imbalance on a large dataset.

MIT's Computer Science and Artificial Intelligence Laboratory (CSAIL) and Microsoft's Mosaic project developed a conditional KNN tree algorithm that groups visually similar artworks from the collections of the Met and the Rijksmuseum in Amsterdam. We also apply a KNN model, but in an unsupervised context and using metadata rather than images.

Although we are aware of no other projects performed on the Met dataset that address a class imbalance problem, ample studies in health, finance, and technology address class imbalance. In "Resampling Imbalanced Network Intrusion Datasets to Identify Rare Attacks" by Sikha Bagui et al, the authors compare a series of strategies for identifying rare cybersecurity attacks. They compare SMOTE, ADASYN, and BSMOTE oversampling techniques to optimize for their selected evaluation metrics, emphasizing the importance of careful attention to resampling methods when the minority class is very rare.

Dataset

The Metropolitan Museum of Art's Open Access collection (https://github.com/metmuseum/openaccess/blob/master/MetObjects.csv) catalogs more than 480,000 artworks in a csv file with 54 features. We dropped columns that we judged to contain duplicate data, based on the definitions on the Met's website, and columns that contained only hyperlinks. For a full list of features and their descriptions, see Appendix 1. We utilized the following columns:

- **`Is_timeline_work`**: work is in the timeline of art history, an institutional decision
- **`Is_highlight`**: work is a curatorial highlight, an institutional decision
- **`Is_public_domain`:** work is in the public domain (not subject to copyright)
- **`Department`**: the department of the Met containing the work
- **`Accession_year`**: the year the Met acquired the work
- **`Object_name`**: Describes the physical type the object is (e.g. painting, vase, dress)
- **`Title`**: Identifying name or phrase given to the work
- **`Portfolio`**: Series or group of published work the piece is a part of
- **`Artist_display_name`:** artist name in the correct order for display

- **`Artist_nationality`:** national, geopolitical, cultural, or ethnic origins of the creator or institution that made the artwork
- **`Artist_gender`:** gender of the artist
- **`Culture`:** information regarding the culture or people the artwork comes from
- **`Country`:** country where the artwork was created or found
- **`Region`:** geographic location; more specific than country, but less specific than subregion
- **`Subregion`:** geographic location, more specific than region
- **`Medium`:** materials used to create the artwork
- **`Tags`**: array of subject keyword tags associated with the object and their respective AAT URL

Pre-Processing

Feature Engineering

Our target variable, `is_significant`, which we constructed, is true if either `is_highlight` or `is_timeline_work` from the original dataset were true. Just 1.935% of values are significant, indicating a highly imbalanced class.

Our feature engineering steps for non-target values were as follows:

- From cleaned and merged data in the `country`, `culture`, `artist nationality`, `region`, `subregion`, and `tags` columns, we constructed **`country_mapped`**.
- We converted `object_begin_date` to **`age`**, the number of years since the work was created. We made this change because the age of the object is more interpretable than a date, particularly when dates include BCE. This choice was also driven by the expectation that models can better parse a feature's importance as a relative value as opposed to a date-type year. This still allows us to bin the values and understand period effects if necessary.
- We converted the `accession_year` column, which contained dates of various format and completeness to a simpler **`access_year`**, the year when the museum acquired the art. (This value did not require the same standardization as `object_begin_date`.)
- We split up the **`tags`** column into a list of tag strings and then vectorized them using a TF-IDF Vectorizer due to the large number of different tags.
- We imputed missing values:
    - For numerical values, our transformer imputes missing values with 0.
    - For categorical values, the transformer imputes missing values with "missing," and then performs one-hot encoding on the non-missing values.
- We scaled all numerical values using StandardScaler().
- We created functions that applied the relevant preprocessing steps to the data for our supervised and unsupervised models:
    - To prepare the supervised model data, the data was split into features (X) and labels (y) datasets. The X dataset was transformed by imputing missing values and performing one-hot encoding. We then use test_train_split to divide those datasets into testing and training sets. Lastly, we created resampled training datasets for both X and y using SMOTE.
    - To prepare the unsupervised model data, the `label` column was dropped to create a feature dataset, X.

Our **final features list** is shown in Appendix 2.

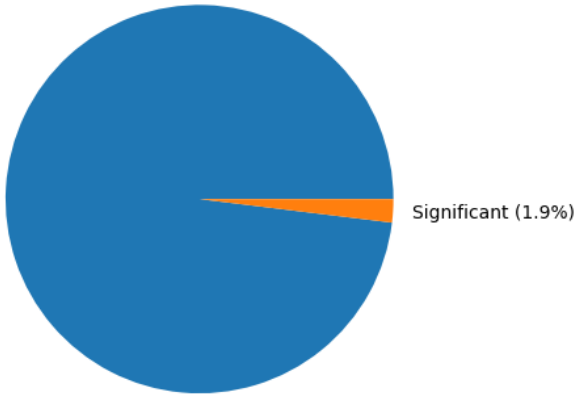
Train-Test Split

Our train-test split was **80-20**, a standard choice for large datasets, retaining sufficient data for training while holding out a meaningful slice for evaluation. We performed the split with **stratification** to ensure that examples from our minority class were included in both splits in the same proportion. This is important in instances of class imbalance, as in our dataset. If the training split has a disproportionate number of examples, the model might perform better on the test split than it would on unseen examples, inflating our metrics and giving us an overly optimistic view of our results.

Resampling Techniques

For the supervised learning analysis, a critical component of our work was addressing class imbalance in our target variable, is_significant, for which the value was True for only 1.935% of rows. To create sufficient data for our model to learn about the minority class, we explored three resampling techniques: synthetic minority oversampling (SMOTE), class weighting, and undersampling.

Figure 1: Class imbalance



SMOTE generates synthetic examples of the minority class by extrapolating from existing examples. While this can improve recall – a critical evaluation metric in our project – it has some drawbacks. SMOTE is not well-suited to mixed-type data. The tags feature is text, and it was not practical to one-hot-encode the tags because of their high cardinality and sparsity. SMOTE carries a higher risk of overfitting than the other techniques, and it is more memory-intensive to implement. Critically, we performed oversampling *after* splitting training and testing data to prevent leakage. The alternative could lead to very similar generated samples ending up in both the training and test sets. This would cause our model to appear to perform better on the test set than it would be likely to in real-world novel examples (Vandeweile et al, 2021).

Class weighting, by contrast, adjusts the model's loss function to penalize misclassification of the minority class more heavily. This approach is more memory-efficient than SMOTE, as it does not generate new samples. Implementation is simple, and it works well on datasets of varying sizes.

Undersampling reduces the size of the majority class to match the minority class. As a result, it is less computationally intensive than SMOTE, but it risks losing diversity and information in the majority class. We can expect to see more variance with undersampling than with class weighting than other techniques, depending on the sample drawn.

Because no single resampling technique was the obvious best choice for our dataset and goals, we applied each strategy to each of our models and compared their performance.

Supervised Learning

Methods

Models

We explored three models: a logistic regression classifier, a random forest classifier, and a linear support vector classifier. We selected these models because of the diversity in their underlying mechanisms and the different advantages each offered for our dataset, which is large, highly imbalanced, and includes both categorical and numerical values.

Logistic regression models the probability of a binary outcome by estimating how each feature contributes to the likelihood of the outcome being true. It is the most interpretable of the three models, and it trains quickly compared to other models, which is useful on our large dataset. Because it does not natively handle categoricals (and nor do the other models we worked with), we added a one-hot-encoding step to preprocessing, as described in that section. We used the same data preparation across all three models.

The **random forest** (RF) classifier is an ensemble method that combines the results of *n* decision trees on random subsets of the training data and outputs the mode of their predictions. It tends to be more accurate than logistic regression, as this aggregate approach reduces variance. Random forest can also better capture complex relationships than logistic regression can. But this comes at the cost of interpretability and increased computational demands, particularly with large numbers of trees.

A **linear support vector classifier** (SVC) establishes the decision boundary between two classes, finding the plane that puts the most distance between them. Similar to random forest, it models complex relationships well and is more computationally demanding than logistic regression. Like logistic regression, it produces a linear decision boundary, but performs better in high-dimensional spaces.

Hyperparameters

For our two best model families, logistic regression and linear SVC, we conducted hyperparameter tuning using randomized search. We selected this method over grid search because it utilizes less processing power; instead of evaluating every possible hyperparameter set, as grid search does, it randomly samples a set number of parameters without replacement. For all models, we limited iterations to 20 parameter sets and we used five-fold cross-validation to get a more stable estimate of model performance across splits. We used stratified sampling to preserve class distribution within each fold, ensuring that our test split also included positives.

We chose to use SMOTE as our resampling technique for both models we conducted hyperparameter tuning on, as it performed best in our preliminary testing. The main parameters we focused on finetuning were `C`, `max_iter`, and `tol` for both models.

Evaluation

Evaluation Metrics

As with model selection, our choice of evaluation metrics was driven by the class imbalance in our target. We could not rely on accuracy as an evaluation metric because a model that predicts False for all values will be correct 99% of the time for our data. Such a model would be useless. Instead, we evaluate based on precision-recall area under the curve (PR-AUC), recall, F1 score, and precision – in that order.

In our experiment, **precision** represents the number of actual significant objects out of all predicted significant objects. If precision is poor, our model will overestimate the significant works. In practice, this would cause our model to generate too many suggestions to curators, which would result in a slower process and potentially lower trust in our model. This is not an ideal outcome, but given that our model's job is to surface potentially significant works, it would be acceptable to have somewhat too many works brought to the curators for review. (If, instead, the model were replacing human selections – which we strongly recommend against – we might instead insist that precision be very high so that few unimportant works are shown to visitors.) While this is the lowest priority of our metrics, we describe it here because it is important to understanding our other key metrics.

Recall, meanwhile, helps us understand the number of true positives, or significant works, our model fails to identify. It is crucial that we surface as many potentially significant works as possible. Therefore, recall weighs more heavily in our model evaluation than precision.

PR-AUC shows the trade-off between precision and recall and tells us how often we correctly identify the minority class. PR-AUC is more relevant than ROC-AUC in imbalanced classes because, like accuracy, ROC-AUC can score high even when the model detects few of the minority class. For our project, this score is the most important.

Because both precision and recall are important, we also look at **F1 score**, (2 x precision x recall)/(precision + recall), which balances the two metrics.

Results

Our results are presented in Figure 2, which shows the five-fold cross-validated results for each set of nine model-resampling-technique pairs. Results for our preferred model are highlighted.

Figure 2: Supervised Learning Model Results

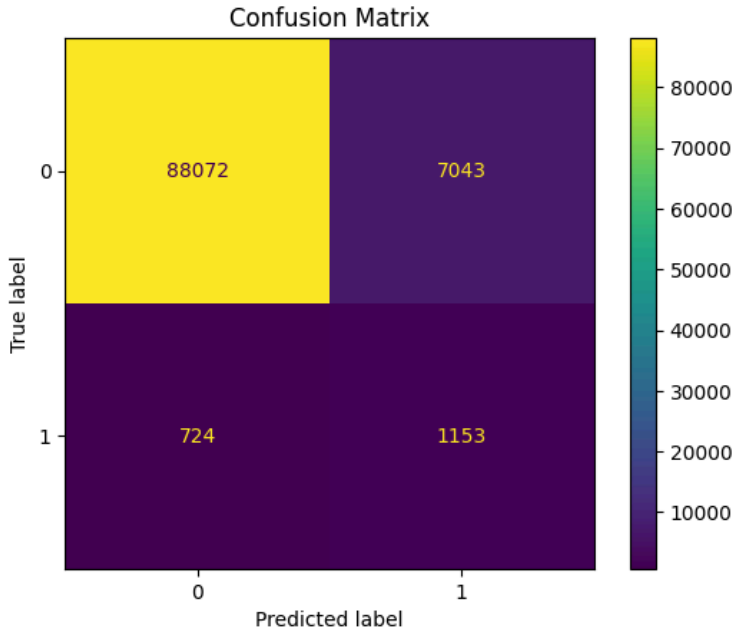
***We include ROC-AUC for illustrative purposes.**

Model and Resampling Technique	Precision mean	Precision STD	Recall mean	Recall STD	F1 mean	F1 STD	PR AUC mean	PR AUC STD	ROC AUC* mean	ROC AUC* STD
LR class weighting	0.5189	0.0304	0.5017	0.0178	0.4571	0.0622	0.0272	0.0180	0.4757	0.1776

LR SMOTE	0.5795	0.0020	0.7467	0.0050	0.6128	0.0029	0.2725	0.0065	0.8977	0.0030
LR undersampling	0.5365	0.0006	0.8105	0.0027	0.5174	0.0019	0.2334	0.0064	0.8926	0.0021
RF class weighting	0.5131	0.0227	0.4932	0.0060	0.4902	0.0067	0.0265	0.0011	0.6160	0.0312
RF SMOTE	0.7664	0.0096	0.5865	0.0005	0.6284	0.0013	0.2990	0.0057	0.8903	0.0018
RF undersampling	0.5400	0.0007	0.8261	0.0013	0.5263	0.0025	0.2266	0.0090	0.9067	0.0017
LSVC class weighting	0.5128	0.0249	0.4770	0.0291	0.4770	0.0327	0.0222	0.0119	0.4256	0.1441
LSVC SMOTE	0.6149	0.0034	0.6258	0.0016	0.6200	0.0013	0.1789	0.0054	0.8273	0.0044
LSVC undersampling	0.5371	0.0006	0.8121	0.0026	0.5192	0.0017	0.1788	0.0032	0.8914	0.0026

Of the strategies we attempted, the logistic regression (LR) classifier paired with SMOTE performed best, achieving the highest PR-AUC score (0.27), the second-highest F1 score (0.61, just behind the best F1 score of 0.62), and strong recall (0.75). These metrics are critical in imbalanced classes, where accuracy and ROC-AUC can be misleading. A PR-AUC of 0.27 is well above random performance for a dataset as imbalanced as ours, in which the positive class is 1.9%. The F1 score suggests a good balance between precision and recall, while the model's low variance across folds suggests its performance is stable. The best parameters for this model were: `C=1`, `max_iter=5000`, `penalty='l2'`, `solver='saga'`, `tol=0.0001`. Results are illustrated in the confusion matrix in Figure 3:

Figure 3: Confusion Matrix



Feature Importance and Ablation Testing

To better understand which features are driving prediction performance in our model, we conducted feature importance analysis and ablation testing. Our logistic regression model's coefficients indicate the direction and strength of each feature's contribution to the prediction (features with .01% importance and greater are shown):

Figure 4: Feature Importance

Feature	Importance
title	0.38%
medium	0.22%
artist_display_name	0.18%
object_name	0.14%
culture	0.05%
artist_nationality	0.02%
portfolio	0.01%

Figure 5: Ablation Testing

Feature dropped	precision	recall	f1	pr_auc	roc_auc
baseline	0.58	0.75	0.61	0.27	0.89
title	0.56	0.78	0.57	0.08	0.78
medium	0.55	0.77	0.56	0.07	0.77
artist_display_name	0.54	0.77	0.54	0.06	0.77

Feature importance analysis revealed that the `title` feature carried the strongest weight and is highly predictive of significance. Cumulative ablation testing confirms this; by dropping the `title` feature, we see a significant decline in performance, indicating that the model relies extremely heavily on it. Other relatively important features, `medium` and `artist_display_name`, also have a noticeable impact on model performance when removed, but the changes are incremental. The high importance of `title` suggests that the model could be overfitting on this feature. On the whole, the model relies heavily on just a few features. In ablation testing, the performance collapses when we remove title, suggesting this single feature is excessively dominant.

Sensitivity Analysis

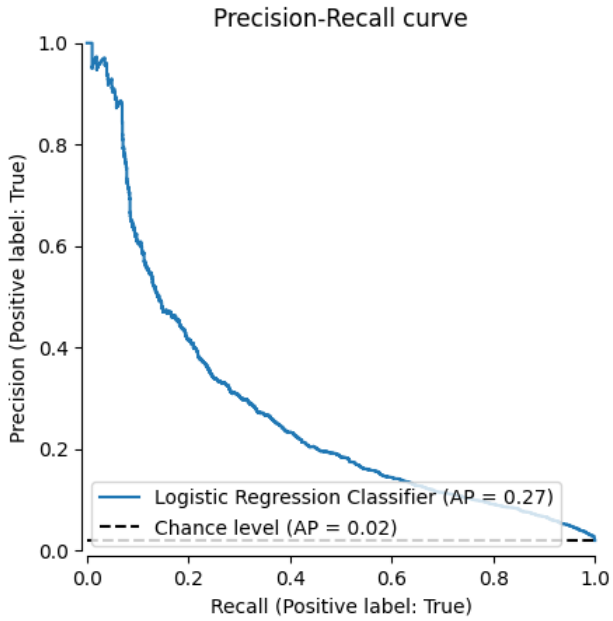
We conducted hyperparameter tuning on our best model by varying the following parameters: `penalty` type (`L1` or `L2`), regularization strength (`C`), solver (`saga`), convergence tolerance (`tol`), and maximum iterations (`max_iter`). We evaluated each configuration across five folds, and the standout was L2 regularization with a strength of `C=1`, a max iteration of `5,000`, and a tolerance of `0.0001`. Across all folds this setup achieved consistent scores, suggesting stable performance across different subsets of the data. Models with low C values, especially those using L1 regularization, underperformed or failed to

converge within the allotted iterations, even with long training times of more than an hour per fold. These two hyperparameters had the largest impact on model performance.

Tradeoffs

The key hyperparameter tradeoffs are between speed and predictive performance as a result of the choice of regularization technique and C value. L1 regularization and low C values converged quickly but performed poorly, while L2 regularization with moderate C values processed slowly but performed well. Among performance metrics, we see the classic tradeoff between precision and recall: as precision increases, recall decreases, and vice-versa, as shown in the precision-recall curve in figure 6:

Figure 6: Precision-Recall Curve



Failure Analysis

We performed a failure analysis to understand the limitations of our model, investigating specific instances of misclassification. In one false negative example, object ID 43951, the “musical stone,” the model incorrectly categorized this item as not significant. This record is **missing two of the three most predictive data points**, `title` and `artist_display_name`. Although `medium` is present, this field alone might not be enough to encode significance for our model. Another false negative, object id 229097, could, on the other hand, fail to classify correctly because it has **too much or overly specific information** in the key fields. In the `medium` feature, we see “Wool, silk (21-23 warps per inch, 8-9 per cm.),” and in the `artist_display_name` field, we see a list of artists’ names who presumably collaborated on the work. The format of the `artist_display_name` field probably does not parse well or reflect more common `artist_display_name` representations in the model.

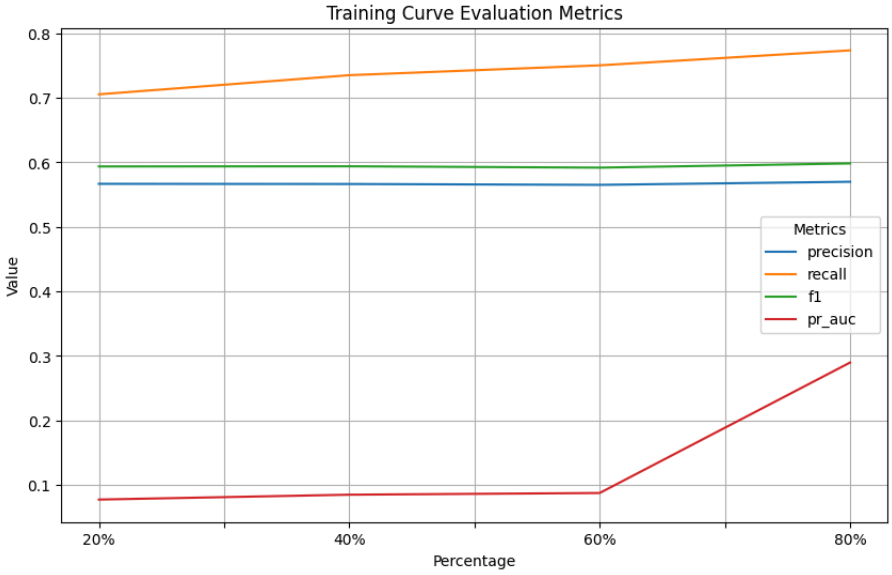
An interesting false positive is object_id 38214, a bust, which has no `title` or `artist_display_name` (or any other artist fields). This suggests that some or all of the extant fields – `department`, `object name`, `country`, `medium`, `culture`, `accession year`, and `age` – are weighing heavily enough in favor of significance to justify the model labeling as such.

This analysis highlights the weight of the top features in the model's decision-making process.This study also suggests the model's understanding of the existing semantics in text fields are limited, and that any textual features should be handled differently with, for example, text embeddings.

Training Curves

Finally, we examined how our model performed with different training sample sizes, using 20, 40, 60, and 80% of the data for training. Other than PR-AUC and recall, our key metrics are stable. That precision and F1 score were stable makes sense because, at every size, our sample is large, and because these other metrics are more focused on classifying the majority class. Meanwhile, PR-AUC and recall improve with larger samples because they especially benefit from seeing more data from the minority class. F1 score does consider recall, but only at one threshold, which is why PR-AUC is more sensitive to more data than F1.

Figure 7: Training Curve Evaluation Metrics



Unsupervised Learning

Methods

Models

We explored a variety of clustering models, but several proved infeasible due to their high computational demands. For example, DBSCAN and HDBSCAN exhausted 128GB of memory in under 30 minutes, making them unsuitable for our dataset. Given the scale and sparsity of the data, we prioritized more efficient approaches. We ultimately selected two models with fundamentally different clustering strategies, allowing us to examine how each would perform on a large, varying population feature dataset while remaining within computational limits.

K-means is a widely used clustering algorithm that prioritizes minimizing the Euclidean distance between data points and their respective centroids. This approach often results in evenly spaced, spherical clusters.This can make k-means less effective at identifying clusters with irregular shapes, and may distort the true structure of our clusters given our mixed population data. However, it is exceptionally fast and scalable, making it well-suited for large datasets like ours. Due to its sensitivity to noise, we applied principal component analysis (PCA) to reduce the dimensionality of the data and focused on the most informative features.

OPTICS is an unsupervised clustering method that differs from k-means in that it does not require specifying the number of clusters in advance. While it typically takes longer to run, OPTICS has the advantage of detecting noise and identifying clusters with varying sizes and densities. This is important for our features, because many of them are much more sparsely populated than others. In most cases, OPTICS is slower than algorithms like DBSCAN or HDBSCAN. However, in our case, the sparse nature and one-hot encoding of some features meant OPTICS could run faster. “The algorithm keeps expanding on a particular point until none of the unprocessed points have a core distance anymore (i.e. aren't core points). These are the outliers” (Versloot 2023) ultimately reducing the number of distance calculations required.

Hyperparameter Tuning

We tuned the hyperparameters of our k-means model by first using the elbow test. We plotted the inertia against the number of clusters and looked for the sharpest bend or elbow, where improvement sharply decreases. Then we looped through `k-means++` and `random` for the `method`, and `10`, `20` for `n_init`. The method and `n_init` that produced the best silhouette scores were chosen for the parameters for our k-means model.

Hyperparameter tuning for the OPTICS model had to be more conservative due to its significantly longer runtime. We adjusted the `min_sample` to include `100`, `1,000`, and `10,000`. We then plotted the clusters, and printed out the characteristics of each cluster. A visual assessment of the plot and characteristics, along with silhouette scores showed a clear performance difference between the three models.

Evaluation

Results

Neither of our methods provided the level of effective clustering we were hoping for, but the k-means algorithm provided far better results than the OPTICS models. We evaluated the effectiveness of our models using visual plot analysis and silhouette scores. Silhouette scores range between -1,1, with -1 as the wrong cluster, and 1 as the best separated clusters possible. We used silhouette score as our primary evaluation technique. It is a commonly used evaluation metric for unsupervised clustering models. Another option we explored was the Davies-Bouldin Index, but that method does better with dense spherical clusters, and our clusters were not spherical and density varied. Silhouette is not perfect as it can be affected by outliers, but works better because it focuses on cohesion and separation.

Table 1: K-Means Hyperparameter Tuning

Initialization Method	Number of Initializations	Silhouette Score
k-means++	**10**	**0.095**
k-means++	20	0.090
random	10	0.088
random	20	0.092

Table 2: OPTICS Tuning

Minimum Sample	Number of Clusters	Silhouette Score
100	903	-0.154
1000	28	-0.161
10000	1	NA

The k-means model surprisingly outperformed the OPTICS models for clustering our dataset. Our best performing k-means cluster scored a 0.095, and our best performing OPTICS cluster scored a -0.154. We did not calculate a silhouette score for the OPTICS model with only one cluster, as the results wouldn't be meaningful as separation of clusters is a main factor in the analysis. The OPTICS model's quicker termination of neighboring points search seems to have eliminated too many points as noise, leading to one OPTICS model only having a single large cluster.

Figure 1a: K-Means Elbow Test

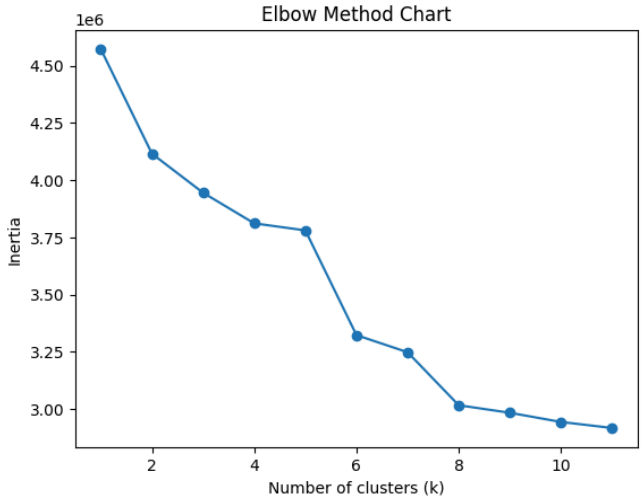


Figure 2a: K-Means Best Model

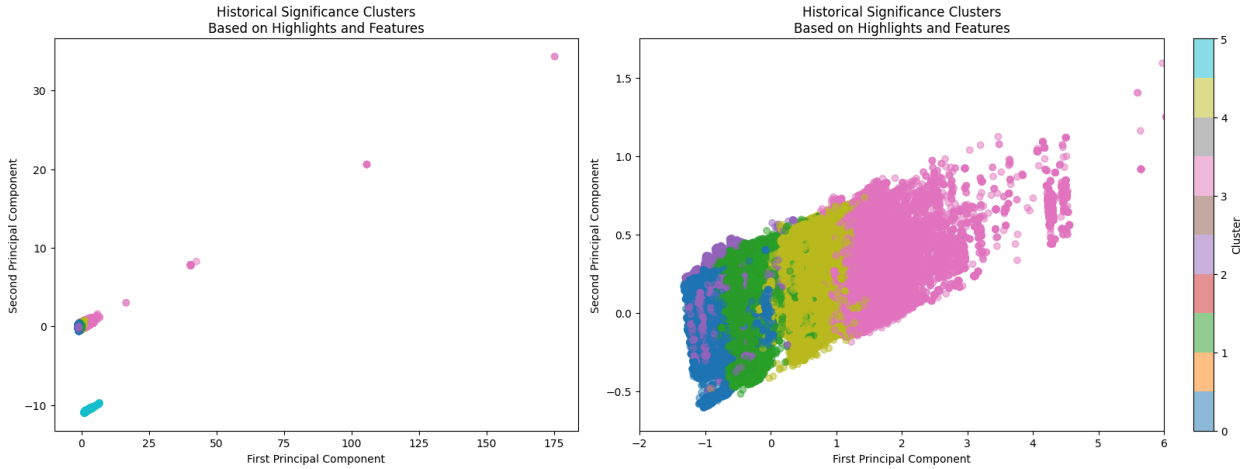


Figure 3a: OPTICS Worst Model

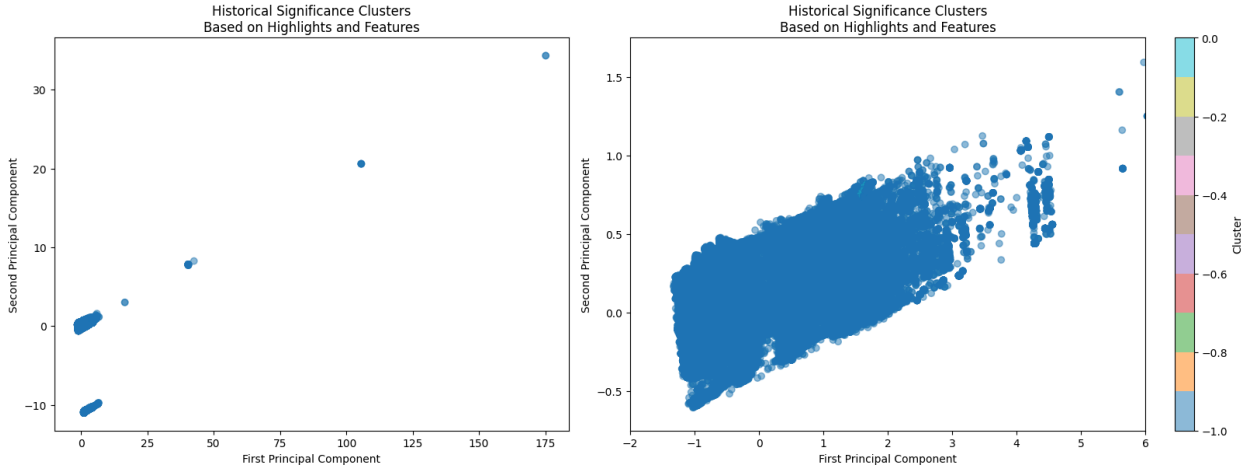
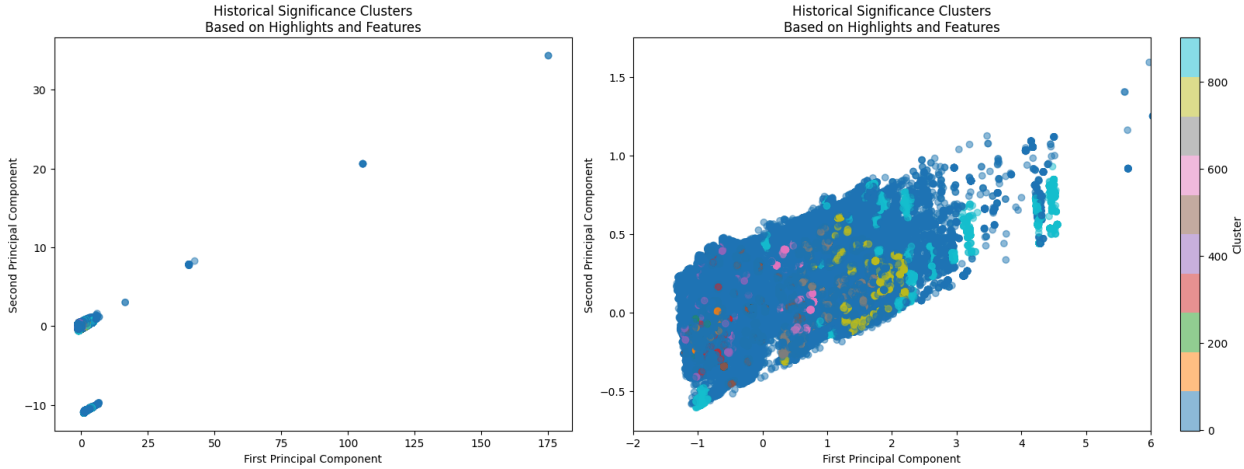


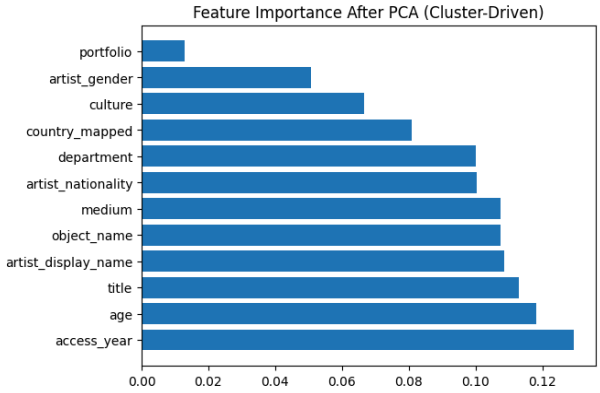
Figure 4a: OPTICS Best Model



We chose features that we felt would offer curatorial guidance and offer distinction between artwork. Then we used PCA to reduce dimensionality, then calculated and extracted the features and their importance

scores to check that no features were overrepresented. We felt our features were well distributed and no features were over- or underrepresented.

Figure 5a: Feature Importance



Sensitivity Analysis

Our k-means model was robust in terms of inertia in our elbow test, and hyperparameter testing. The variation between models in our hyperparameter tuning was only between 0.088-0.95. The inertia in our elbow test only varied between roughly 4.2 and 3 le6 (Figure 1a). While our results were not what we were hoping for, our model was stable and not dependent on a single element.

Discussion

In our supervised learning exploration, we were surprised that the best-performing model was the logistic regression classifier; we expected that either of the other algorithms, which generally better model complex relationships between features, would perform better. In our unsupervised learning study, we were surprised that the k-means models created better-defined clusters than OPTICS. We expected the OPTICS model to perform better due to its ability to handle non-spherical clusters, but its performance was unexpectedly poor. We believe that may have been due to some of our features being sparsely populated and the OPTICS algorithm terminating neighbor searches quickly if they were too far apart.

Computational resources were a challenge at various points in the project. For example, training the random forest classifier was resource-intensive, as was training the OPTICS model. We ultimately used DeepNote to train the OPTICS model. Similarly, as we noted in our supervised modeling results, some hyperparameter sets processed very slowly.

Future research could extend this work in several ways. To improve performance of our supervised model, we would tune parameters for our resampling methods, and we would experiment with model ensembling. For both supervised and unsupervised approaches, we would reintroduce the tags field from the metadata to provide additional signal. Most importantly, the project would benefit from review by a curatorial expert who could share intuition about our goals and our findings and help us refine our approach to modeling curatorial decisions.

Ethical Considerations

One of our project's goals is to understand the implicit logic underlying the Met's designations of certain artworks as significant. Although we are not making a value judgment on the institution's curatorial choices, we recognize that modeling them has important ethical implications.

Any model trained on the museum's existing designations will inherit and reinforce the biases present in the original labels. Biases could include preferences toward already culturally dominant groups across dimensions, such as artist language, artist gender, artist race, object country/continent of origin, time period, etc. It is crucial to acknowledge that our models do not diversify or update existing definitions of significance, but reflect and perpetuate the patterns already established.

Specifically with respect to our unsupervised learning analysis, clustering presents additional risks. Clustering methods look for similarities, and can thereby reinforce the human bias implicit in the clusters. For example, groupings like European and non-European art might reflect institutional conventions, not objective similarities or differences. Clustering can also erase underrepresented classes, as clustering by its nature is looking for commonalities at the expense of nuance.

Similarly, we must be careful to convey our findings as reflections of the subjective choices of a single institution. The labels "is timeline work" and "in timeline of art history" are not objective markers of significance, nor are they universally or even broadly agreed upon as significant. If these distinctions are presented without this context, there is a risk that our audience will come away with a distorted understanding of art history.

Finally, we also acknowledge that designating works as a highlight or in the timeline of history might not be a strictly academic exercise. These labels could be influenced by factors not available in the dataset, such as donor preferences, internal institutional politics, broader politics and culture, and current events.

Statement of Work

While all of us worked on most, if not all, aspects of our project, we use this section to describe who did the heavy lifting on each component.

Code: Molly wrote our code, including leveraging work that Molly and Sarah completed as part of Milestone 1 for initial data cleaning steps, and organized our git repository.
Research: Sarah researched many and various questions that arose during our project, and Sam researched likely best models and strategies for an imbalanced target class.
Visualizations: Sarah and Molly created our visuals.
Report: Sam and Sarah wrote the report.
Meetings: Sam organized our meetings.

Bibliography

Bagui, Sikha, and Kunqi Li. 2021. “Resampling Imbalanced Data for Network Intrusion Detection Datasets.” *Journal of Big Data* 8 (1). https://doi.org/10.1186/s40537-020-00390-x.

Sattiraju, Sarath. 2018. “Museum-Ml/the Metropolitan Museum of Art Open Access.ipynb at Master · Sarathisme/Museum-Ml.” GitHub. 2018. https://github.com/Sarathisme/museum-ml/blob/master/The%20Metropolitan%20Museum%20of%20Art%20Open%20Access.ipynb.

Stark, Molly, and Sarah Moore. 2025. “GitHub - Mollykstark/SIADS-Milestone-1: University of Michigan School of Information Applied Data Science Milestone I.” GitHub. 2025. https://github.com/mollykstark/SIADS-Milestone-1/tree/main.

“The Metropolitan Museum of Art Collection API.” 2020. The Metropolitan Museum of Art Collection API. November 17, 2020. https://metmuseum.github.io/.

Vandewiele, Gilles, Isabelle Dehaene, György Kovács, Lucas Sterckx, Olivier Janssens, Femke Ongenae, Femke De Backere, et al. 2021. “Overly Optimistic Prediction Results on Imbalanced Data: A Case Study of Flaws and Benefits When Applying Over-Sampling.” *Artificial Intelligence in Medicine* 111 (Special Issue): 101987. https://doi.org/10.1016/j.artmed.2020.101987.

Versloot, Christian. 2023. “Performing-Optics-Clustering-With-Python-And-Scikit-Learn.md.” GitHub. 2023. https://github.com/christianversloot/machine-learning-articles/blob/main/performing-optics-clustering-with-python-and-scikit-learn.md.

Appendix

Appendix 1: Complete features list from the Met

Object Number
Is Highlight
Is Timeline Work
Is Public Domain
Object ID
Gallery Number
Department
AccessionYear
Object Name
Title
Culture
Period
Dynasty
Reign

Portfolio
Constituent ID
Artist Role
Artist Prefix
Artist Display Name
Artist Display Bio
Artist Suffix
Artist Alpha Sort
Artist Nationality
Artist Begin Date
Artist End Date
Artist Gender
Artist ULAN URL
Artist Wikidata URL
Object Date
Object Begin Date
Object End Date
Medium
Dimensions
Credit Line
Geography Type
City
State
County
Country
Region
Subregion
Locale
Locus
Excavation
River
Classification
Rights and Reproduction
Link Resource
Object Wikidata URL
Metadata Date
Repository
Tags
Tags AAT URL
Tags Wikidata URL

Appendix 2: Final features list

- is_significant
- is_public_domain
- department
- access_year
- object_name
- title
- culture
- portfolio
- artist_display_name

- artist_nationality
- artist_gender
- country_mapped: Country art originates from, based on cleaned, merged data from the country, culture, artist nationality, region, subregion, and tags columns
- age
- medium
- tags